EX 3: COMPARISON OF SGD WITH MOMENTUM VS ADAM OPTIMIZER

AIM

To implement and compare Stochastic Gradient Descent (SGD) with momentum and the Adam optimizer using a Multi-Layer Perceptron (MLP) on the CIFAR-10 dataset, and analyze their convergence rates and performance using loss and accuracy.

PROCEDURE

1. Import the required libraries, including PyTorch, torchvision, and Matplotlib.
2. Select the appropriate computing device (CPU or GPU) and load the CIFAR-10 dataset.
3. Preprocess the images by converting them into tensors and normalizing their pixel values.
4. Define an MLP model with an input layer, a hidden layer containing 256 neurons with ReLU activation, and an output layer containing 10 neurons.
5. Define a training function using cross-entropy loss and backpropagation.
6. Train the first model using SGD with a learning rate of 0.01 and momentum of 0.9.
7. Train the second model using the Adam optimizer with a learning rate of 0.001.
8. Record the training loss and accuracy for each epoch for both models.
9. Plot and compare the loss and accuracy curves to analyze the convergence behavior of the two optimizers.

CODE

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt


# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)


# Load and preprocess CIFAR-10 dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])
```

```
trainset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    transform=transform,
    download=True
)

trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=128,
    shuffle=True
)

# Define MLP model
class MLP(nn.Module):
    def __init__(self):
        super().__init__()

        self.flatten = nn.Flatten()

        self.net = nn.Sequential(
            nn.Linear(3 * 32 * 32, 256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.net(x)

# Training function
def train(model, optimizer, epochs=10):
```

```
model.to(device)
loss_fn = nn.CrossEntropyLoss()

losses = []
accuracies = []

for epoch in range(epochs):
    total_loss = 0
    correct = 0
    total = 0

    model.train()

    for imgs, labels in trainloader:
        imgs = imgs.to(device)
        labels = labels.to(device)

        outputs = model(imgs)
        loss = loss_fn(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    avg_loss = total_loss / len(trainloader)
    accuracy = 100.0 * correct / total
```

```
        losses.append(avg_loss)
        accuracies.append(accuracy)

        print(
            f"Epoch {epoch + 1}: "
            f"Loss = {avg_loss:.4f}, "
            f"Accuracy = {accuracy:.2f}%"
        )

    return losses, accuracies

# Train using SGD with Momentum
model_sgd = MLP()
sgd = optim.SGD(
    model_sgd.parameters(),
    lr=0.01,
    momentum=0.9
)

print("\nTraining with SGD + Momentum")
losses_sgd, acc_sgd = train(model_sgd, sgd)

# Train using Adam
model_adam = MLP()
adam = optim.Adam(
    model_adam.parameters(),
    lr=0.001
)

print("\nTraining with Adam")
losses_adam, acc_adam = train(model_adam, adam)

# Visualize loss and accuracy comparison
```

```
plt.figure(figsize=(12, 5))

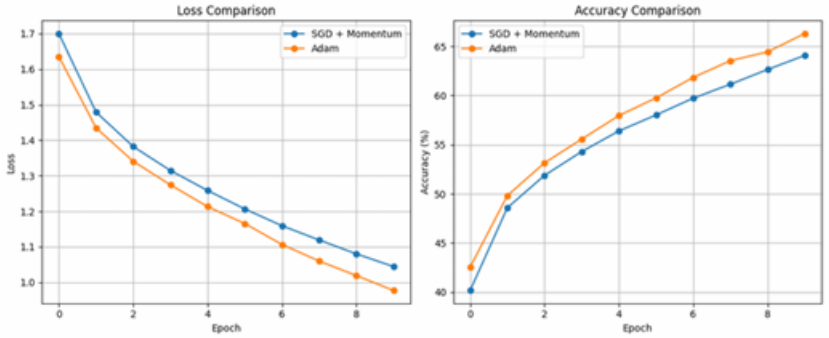
plt.subplot(1, 2, 1)
plt.plot(losses_sgd, marker='o', label="SGD + Momentum")
plt.plot(losses_adam, marker='o', label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss Comparison on CIFAR-10")
plt.legend()
plt.grid(True)

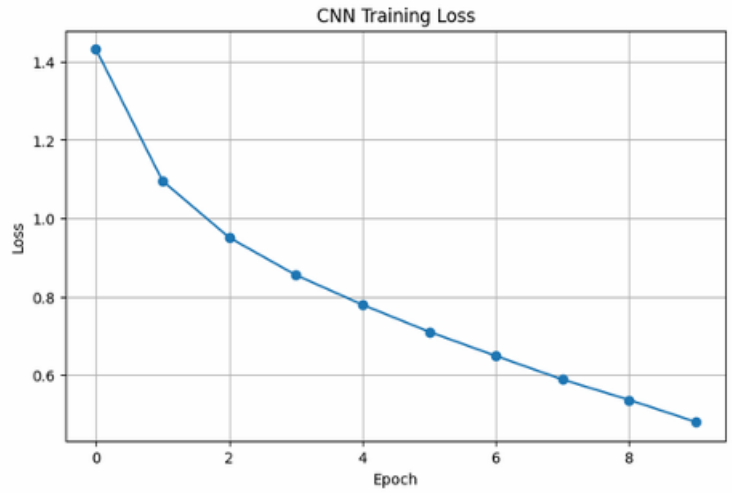
plt.subplot(1, 2, 2)
plt.plot(acc_sgd, marker='o', label="SGD + Momentum")
plt.plot(acc_adam, marker='o', label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy Comparison on CIFAR-10")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

OUTPUT

```
Final Results:
 SGD + Momentum - Loss: 1.0445, Accuracy: 64.08%
Adam - Loss: 0.9773, Accuracy: 66.30%
```





RESULT

Thus, SGD with momentum and Adam optimizers were successfully implemented using PyTorch and evaluated on the CIFAR-10 dataset. Their training loss and accuracy were recorded and visualized to compare their convergence behavior. Adam generally provides faster initial convergence due to its adaptive learning rates, while SGD with momentum can offer competitive performance depending on the learning rate, model architecture, and training duration.